PENNYWISE
EXPENSE TRACKER & BUDGET MANAGEMENT SYSTEM

A Project-Based Learning (PBL) Case Study and Python Project

Submitted in partial fulfilment of the requirements for the subject: Python Programming

Submitted by: _____
Roll Number: _____
Class / Section: _____
Department: _____
University / College: _____

Submitted to: _____
Academic Session: 2026–2027

Project implementation: Python, Streamlit, SQLite, Pandas and Plotly

CERTIFICATE

This is to certify that the Project-Based Learning (PBL) report entitled “PennyWise — Expense Tracker & Budget Management System” is a bona fide work carried out by _____, Roll Number _____, of _____, under my guidance and supervision during the academic session 2026–2027.

The work presented in this report is submitted in partial fulfilment of the requirements of the Python Programming subject. To the best of my knowledge, the work is original and has not been submitted elsewhere for an academic award.

Project Guide: _____
Signature: _____
Date: _____

Head of Department: _____
Signature / Seal: _____

DECLARATION

I hereby declare that the project entitled “PennyWise — Expense Tracker & Budget Management System” has been developed by me as part of the Project-Based Learning activities for Python Programming. The design, implementation, testing and documentation have been completed for educational purposes.

I have made reasonable efforts to acknowledge the tools, libraries and learning resources used. The application is a classroom prototype and is not represented as a certified financial product or a substitute for professional financial advice.

I understand that I am responsible for explaining the source code, demonstrating the application and making any corrections suggested by my faculty guide.

Student Name: _____
Roll Number: _____
Signature: _____
Date: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my Python Programming faculty and project guide for their guidance, feedback and encouragement during the development of this PBL project. Their suggestions helped me convert a common personal-finance problem into a structured software solution.

I also thank my department and institution for providing the academic environment and resources required to complete the project. I acknowledge the developers and documentation communities behind Python, Streamlit, SQLite, Pandas and Plotly. Their publicly available documentation made it possible to learn the relevant concepts and build the prototype.

Finally, I thank my classmates, friends and family members for their suggestions about everyday expense tracking and budget planning. Their feedback helped shape the categories, dashboard and workflow of PennyWise.

ABSTRACT

PennyWise is a simple expense tracking and budget management application developed using Python. The project addresses a familiar problem: people make many small purchases during a month but often do not maintain a clear record of where their money goes. Without organised records, it becomes difficult to compare spending against a budget, identify high-cost categories or make informed adjustments.

The application provides a central interface to record expenses with a description, amount, category, date, payment method and optional note. It stores information in a local SQLite database. A dashboard calculates monthly and all-time spending, displays category and daily-spending charts, and compares actual expenditure with user-defined monthly category limits. The system highlights when a budget reaches 80 percent and when it is exceeded. Users can search transactions, delete an incorrect record and export filtered data to CSV.

The project demonstrates practical use of Python functions, validation, database operations, data analysis and visualisation. Streamlit is used for the user interface, Pandas for tabular analysis and Plotly for charts. The result is an accessible educational prototype that can be extended with authentication, cloud synchronisation and recurring transactions.

TABLE OF CONTENTS

1. Introduction
2. Case background and problem statement
3. Objectives and scope
4. Existing system and proposed solution
5. Feasibility study
6. Requirement analysis
7. System design and architecture
8. Database design
9. Algorithms and workflow
10. Technology and Python concepts
11. Implementation details
12. User interface and modules
13. Testing strategy and test cases
14. Results and discussion
15. Security, privacy and limitations
16. Project planning and risk management
17. Future enhancements
18. Conclusion
19. Bibliography
20. Appendix A — Installation and execution
21. Appendix B — GitHub publication
22. Appendix C — Viva questions

1. INTRODUCTION

Personal financial management is the process of planning, recording and reviewing income and expenditure. Students and young professionals often use digital payments, cash and cards across many small purchases. The convenience of these payment methods can make spending less visible. A person may remember large purchases but forget minor daily expenses such as snacks, transport or subscriptions. These small transactions can collectively affect the monthly budget.

An expense tracker reduces this information gap by keeping a consistent record. A budget adds a planning dimension: it defines a spending limit before money is spent and allows actual spending to be compared with that limit. When both functions are combined, users can understand not only how much they spent but also whether their spending is aligned with their plan.

PennyWise is designed as a beginner-friendly desktop/web-local application. It focuses on the essential workflow rather than complex accounting. It does not connect to bank accounts or process payments. The user manually enters transactions, chooses a category and reviews the resulting dashboard. This limited scope keeps the project understandable while demonstrating important Python programming skills.

2. CASE BACKGROUND & PROBLEM STATEMENT

Case scenario: A college student receives a fixed monthly allowance. During the month, the student pays for meals, travel, mobile recharge, stationery, entertainment and occasional shopping. Some payments are made through UPI, some in cash and others by card. At the end of the month, the student finds it difficult to explain why the available balance is lower than expected. Receipts may be missing and notes stored in different places are inconsistent.

The student tries to control spending by remembering approximate amounts. This approach is unreliable because memory is not a structured transaction record. It also does not provide a category-wise view or an early warning when a spending limit is nearly reached.

Problem statement: There is a need for a simple, low-cost application that allows a user to record everyday expenses, organise them by category, define monthly budget limits and view understandable summaries. The system should reduce manual calculation, make records searchable and support export for review.

Stakeholders include the primary user, the faculty evaluator and a future developer who may extend the prototype. The primary user needs clarity and ease of use; the evaluator needs evidence of programming concepts, database use and testing; a future developer needs readable code and setup documentation.

3. OBJECTIVES AND SCOPE

Primary objectives:
• Create a simple interface for recording expense details.
• Store transactions persistently using SQLite.
• Categorise expenses to support meaningful analysis.
• Calculate monthly and all-time expenditure.
• Allow monthly budget limits to be set per category.
• Display charts and clear budget-status messages.
• Search, review, delete and export transaction records.

In-scope functions: manual expense entry, category selection, date selection, payment method selection, local storage, dashboard summaries, monthly budgets, transaction filtering, CSV export and deletion of a selected transaction.

Out-of-scope functions: bank account integration, automatic SMS reading, payment processing, tax filing, multi-user cloud synchronisation, investment advice and production-grade financial compliance. The prototype does not calculate a bank balance because it does not record income or opening balance in its current version.

Success criteria: a user can install dependencies, launch the application, add an expense, see it in the transaction history, define a budget, view a chart and export records without manually editing the database. Input validation should reject blank descriptions and non-positive amounts.

4. EXISTING SYSTEM & PROPOSED SOLUTION

In an informal existing process, a user may rely on memory, paper notes, a calculator or a general spreadsheet. Memory is convenient but inaccurate. Paper notes can be lost and require manual totals. A spreadsheet offers flexibility but may require the user to build formulas and charts. These methods can work, but they do not necessarily provide a guided transaction-entry workflow or immediate budget warnings.

The proposed PennyWise system provides a single interface for the core tasks. The user enters a transaction through a form. Python validates the values and writes the record to SQLite. The dashboard reads the stored data and uses Pandas to group spending by month, date and category. Plotly turns grouped results into charts. Budget records are compared with actual category totals and displayed as progress indicators.

The proposed solution is intentionally small. It avoids bank credentials and third-party financial APIs, which keeps the classroom demonstration safer and easier to reproduce. Its local database also means that the user controls the project folder. However, local storage requires backups and does not automatically synchronise between devices.

5. FEASIBILITY STUDY

Technical feasibility: The project uses widely available Python libraries. Streamlit provides forms and navigation; SQLite is included with Python; Pandas handles data tables; Plotly creates interactive charts. A standard laptop capable of running Python can execute the application. No dedicated server is required for local demonstration.

Economic feasibility: All selected tools are open source or freely available for educational use. The application does not require paid hosting, a commercial database or payment gateway. The main cost is development time and access to a computer.

Operational feasibility: The interface is organised into Dashboard, Add Expense, Manage Budget, Transactions and About sections. Labels use familiar financial terms. The user does not need to know SQL to enter or review data. CSV export allows data to be opened in spreadsheet software.

Schedule feasibility: The work can be completed in stages: requirements, database design, interface, calculations, charts, testing and documentation. A small feature set makes it suitable for a semester PBL assignment.

Legal and ethical considerations: The project should use sample or personal test data only. It should not collect bank passwords, card numbers or authentication secrets. If deployed publicly, data protection, access control and secure hosting would need further design.

6. REQUIREMENT ANALYSIS

Functional requirements:
FR1 — The system shall accept an expense description and positive amount.
FR2 — The system shall allow a category, date and payment method to be selected.
FR3 — The system shall save each expense with a unique ID.
FR4 — The dashboard shall calculate monthly and all-time totals.
FR5 — The system shall group monthly spending by category and date.
FR6 — The user shall be able to save or update a category budget for a month.
FR7 — The system shall show progress and warnings when a budget is near or above its limit.
FR8 — The user shall search, view, delete and export transactions.

Non-functional requirements: usability, readable code, basic input validation, data persistence, responsive layout, reasonable performance for a student-sized dataset and reproducible installation. The application should show an understandable message when no records exist.

Inputs include text, decimal amounts, dates, categories and payment methods. Outputs include summary metrics, charts, budget status, transaction tables and CSV files. Constraints include a positive expense amount, non-empty title, valid category and a date supplied by the date picker. SQLite enforces a positive amount through a CHECK constraint.

7. SYSTEM DESIGN & ARCHITECTURE

The application follows a simple three-part logical design. The presentation layer is the Streamlit interface. It collects user input and displays tables, metrics, progress bars and charts. The application logic layer contains Python functions for database connection, initialisation, inserting expenses, reading records, deleting records and saving budgets. The data layer is a local SQLite file.

Data flow for a new expense: user opens Add Expense → enters details → submits form → validation checks description and amount → add_expense() executes a parameterised INSERT statement → SQLite stores the record → Streamlit reruns → dashboard and transaction history read the updated data.

Data flow for budget analysis: dashboard loads expenses → filters rows to the current month → groups amounts by category → reads budget rows for the month → joins actual totals to limits → calculates utilisation → displays progress and warning status.

The architecture keeps database operations in small functions so that the interface code does not repeat SQL statements. Parameterised queries are used for user-supplied values. The prototype is single-user and local; it does not include a separate API server or user authentication.

8. DATABASE DESIGN

The database contains two tables: expenses and budgets.

EXPENSES: id INTEGER PRIMARY KEY AUTOINCREMENT; title TEXT NOT NULL; amount REAL NOT NULL CHECK(amount > 0); category TEXT NOT NULL; spent_on TEXT NOT NULL; payment_method TEXT NOT NULL; note TEXT DEFAULT ''. Each row represents one recorded expense. The ISO date format YYYY-MM-DD makes month filtering straightforward.

BUDGETS: id INTEGER PRIMARY KEY AUTOINCREMENT; month TEXT NOT NULL; category TEXT NOT NULL; limit_amount REAL NOT NULL CHECK(limit_amount > 0); UNIQUE(month, category). The combination of month and category is unique, so saving an existing category budget updates its limit instead of creating a duplicate.

Relationship: there is no enforced foreign key between the tables because a budget is a monthly category target and expenses are transaction records. The application joins aggregated expense totals with budget rows using the category field during dashboard calculation.

SQLite is appropriate for a local PBL prototype because it is file-based and requires no separate database server. For a multi-user hosted application, a server database and authentication would be more suitable. A production financial system should also consider decimal-safe currency storage, audit logs, backups and access policies.

9. ALGORITHMS & WORKFLOW

Algorithm — Add expense:
1. Display the expense form.
2. Read title, amount, category, date, payment method and note.
3. If title is blank, display an error and stop.
4. If amount is not greater than zero, display an error and stop.
5. Insert the validated values using a parameterised SQL statement.
6. Display success and refresh the application.

Algorithm — Monthly spending:
1. Read all expense rows.
2. Create current month key in YYYY-MM format.
3. Select records whose date begins with that key.
4. Sum the amount column.
5. Group the selected rows by category and date for charts.

Algorithm — Budget status:
1. Read budget limits for the selected month.
2. Calculate actual spending by category.
3. Match actual totals to each budget category.
4. Utilisation = actual spending ÷ budget limit.
5. If actual exceeds limit, show exceeded amount.
6. Else if utilisation is at least 0.80, show warning.
7. Otherwise show within-budget status.

The progress display is capped at 100 percent so that an over-budget value does not exceed the valid progress-bar range. The text warning still reports the actual excess.

10. TECHNOLOGY & PYTHON CONCEPTS

Python is the main programming language. Functions such as connect(), init_db(), add_expense(), get_expenses(), delete_expense(), save_budget() and get_budgets() divide the application into reusable tasks. Conditional statements validate inputs and decide which dashboard messages should be displayed. Loops process budget rows and display status for each category.

SQLite provides persistent storage through SQL statements. CREATE TABLE defines the schema, INSERT stores records, SELECT retrieves them and DELETE removes a selected transaction. The ON CONFLICT clause updates an existing monthly category budget.

Streamlit converts Python code into an interactive web interface. Forms group inputs and submit them together. Sidebar radio buttons provide navigation. Metrics, dataframes, progress bars and download buttons present information.

Pandas provides DataFrame structures, filtering, grouping, aggregation and CSV conversion. Plotly Express creates a doughnut-style category chart and a daily-spending bar chart. The combination demonstrates how Python can connect user interaction, storage, data analysis and visualisation in one project.

11. IMPLEMENTATION DETAILS

The application begins by configuring the Streamlit page and calling init_db(). The database initialisation function creates the two tables only if they do not already exist, allowing the application to be restarted without losing existing records.

The Add Expense form collects a description, amount, category, date, payment method and optional note. The number input has a positive minimum, while an additional validation check ensures the title is not blank. The database also applies a CHECK constraint to the amount.

The dashboard reads expense rows into a Pandas DataFrame. It calculates the current month key and filters dates using the YYYY-MM prefix. The total is computed using the amount column. If data exists, groupby() creates category and daily totals. If no data exists, the interface displays a helpful empty-state message rather than an empty chart.

The budget page stores one limit for each month/category combination. The transaction page supports case-insensitive search on title and category, CSV download and deletion by selected ID. The application refreshes after successful write operations so the interface reflects the latest database state.

12. USER INTERFACE & MODULES

Dashboard: Shows current-month expenditure, all-time expenditure and total transaction count. It displays a category chart, daily spending chart, recent transactions and budget status when data is available.

Add Expense: Provides a form for the transaction description, amount, category, date, payment method and optional note. A successful submission stores the record and clears the form.

Manage Budget: Lets the user select a target month, category and positive monthly limit. Saving the same month/category combination updates the existing limit.

Transactions: Shows transaction history in descending date order. The search box filters by description or category. A CSV download button exports the currently filtered rows. A selected transaction can be deleted.

About: Explains the purpose, features, technology and educational limitations of the project.

The visual design uses Streamlit's standard components to keep the interface consistent and easy to maintain. During the demonstration, the evaluator should first add sample expenses, then set a budget and return to the dashboard to observe the charts and warning logic.

13. TESTING STRATEGY & TEST CASES

Testing is performed manually through the interface and by checking the database results. The following cases should be demonstrated:

TC01 Valid expense: enter “Lunch”, ■120, Food and today's date. Expected: success message and row appears.
TC02 Blank description: submit with an empty title. Expected: validation error; no row inserted.
TC03 Zero/negative amount: attempt invalid amount. Expected: UI prevents it; database constraint also rejects non-positive values.
TC04 Category analysis: add expenses in Food and Transport. Expected: category chart contains both categories and correct totals.
TC05 Monthly total: add two expenses in the current month. Expected: dashboard total equals their sum.
TC06 Budget under 80%: set ■1,000 and spend ■500. Expected: within-budget message.
TC07 Budget at 80%: spend ■800. Expected: warning message.
TC08 Budget exceeded: spend ■1,100. Expected: exceeded message reports ■100.
TC09 Search: search for “Food”. Expected: matching category rows only.
TC10 CSV export: download transaction list. Expected: valid CSV with column headings.
TC11 Delete: delete a selected ID. Expected: row disappears after refresh.
TC12 Empty state: run with no expenses. Expected: informative message, no application crash.

Testing should be repeated after changing the month selector and after restarting the app to verify database persistence.

14. RESULTS & DISCUSSION

The expected outcome is a functioning local application that completes the core expense-management workflow. A user can add a transaction, store it, view it in a table, analyse monthly totals and compare category spending with budgets. The charts convert transaction rows into a visual summary that is easier to interpret than a list of amounts.

The project demonstrates the value of structured records. Instead of manually adding expenses, the application calculates totals from stored transactions. Category grouping can reveal which areas account for the largest share of monthly spending. Budget progress can prompt the user to review a category before the limit is crossed.

The quality of the insight depends on the quality of the data. If users forget to enter cash purchases or choose inconsistent categories, the totals will be incomplete or misleading. The application therefore supports manual record keeping but cannot guarantee that every real-world transaction has been captured.

For academic evaluation, evidence can include screenshots of the dashboard, add-expense form, budget page, transaction search and CSV export. Use sample data and explain the calculations during the live demonstration.

15. SECURITY, PRIVACY & LIMITATIONS

The project uses SQLite locally and does not ask for bank login credentials, card numbers or UPI PINs. SQL statements that include user-provided values use placeholders rather than string concatenation, reducing SQL injection risk. The application is intended for local educational use.

Important limitations: there is no authentication, so anyone with access to the project folder may access the database. There is no cloud backup or device synchronisation. The current schema stores amount as REAL for simplicity; production financial applications should use integer minor units (paise) or a decimal-safe representation to avoid floating-point rounding issues. The app does not record income, opening balance, transfers, recurring expenses or shared household budgets.

Before public deployment, the project would require authentication, authorisation, secure hosting, protected database access, backups, input-size limits, logging and a privacy policy. It should not be presented as a banking application. Users should keep backups of the SQLite file if they want to preserve records.

16. PROJECT PLAN & RISK MANAGEMENT

Suggested development plan:
Week 1 — Select the problem, define users and collect requirements.
Week 2 — Design the database schema and user workflow.
Week 3 — Build expense entry and transaction history.
Week 4 — Add monthly budgets and calculations.
Week 5 — Add charts, search and CSV export.
Week 6 — Test, correct defects, prepare screenshots and documentation.

Risks and responses: dependency installation failure — use a virtual environment and requirements file; database file accidentally deleted — keep a backup; invalid input — validate in the interface and database; incorrect monthly totals — use ISO dates and test month boundaries; chart fails with empty data — display an empty-state message; GitHub accidentally receives private data — ignore the SQLite database and never commit credentials.

A simple version-control approach is to commit after each working milestone. Commit messages should describe the change, for example “Add expense entry form” or “Implement monthly budget warnings”. This makes the development history easier to understand and allows changes to be reviewed.

17. FUTURE ENHANCEMENTS

The project can be extended in several directions. User authentication would allow separate personal accounts. A hosted database could synchronise records across devices. Income and opening-balance modules could provide a more complete cash-flow view. Recurring expenses could automatically create monthly entries after user confirmation. Receipt uploads and OCR could reduce manual typing, although the extracted amount and date should always be confirmed by the user.

Additional analytics could include month-to-month comparisons, average daily spending, savings goals, category trends and budget recommendations based on previous months. A mobile-friendly interface or installable progressive web app could improve accessibility. Notifications could warn users when a category approaches its limit.

For a more robust implementation, the application could separate its business logic from the Streamlit interface, introduce automated unit tests, use integer paise for currency, add database migrations and create encrypted backups. These enhancements should be prioritised according to user need, privacy and project time rather than added only for complexity.

18. CONCLUSION

PennyWise demonstrates how Python can be used to solve a practical everyday problem through a small but complete software application. It combines a guided interface, persistent database storage, data processing, visualisation and budget comparison. The project converts manually recorded transactions into organised information that can support better awareness of spending habits.

The objectives of recording expenses, categorising transactions, calculating monthly totals, setting budgets, showing warnings and exporting records are addressed in the prototype. The work also provides practical experience with functions, validation, SQL, Pandas DataFrames and interactive charts.

The project has deliberate limitations: it is local, single-user and manually updated. It is not connected to financial institutions and should not be used to store sensitive credentials. These limitations create clear opportunities for future development while keeping the current PBL project achievable and explainable.

In conclusion, the project meets its educational purpose by demonstrating a complete Python application lifecycle: problem identification, requirement analysis, design, implementation, testing, documentation and publication through GitHub.

19. BIBLIOGRAPHY

1. Python Software Foundation, “Python Documentation”, https://docs.python.org/3/ (accessed for language and standard-library concepts).
2. Streamlit, “Streamlit Documentation”, https://docs.streamlit.io/ (accessed for forms, navigation, metrics and download controls).
3. SQLite, “SQLite Documentation”, https://www.sqlite.org/docs.html (accessed for SQL and database concepts).
4. pandas, “pandas Documentation”, https://pandas.pydata.org/docs/ (accessed for DataFrame operations, grouping and CSV export).
5. Plotly, “Plotly Python Documentation”, https://plotly.com/python/ (accessed for interactive chart concepts).
6. GitHub Docs, “Getting started with Git”, https://docs.github.com/en/get-started/using-git (accessed for repository publication workflow).

Note: Add any textbooks, lecture notes or additional learning resources actually used during your implementation. Follow the citation style prescribed by your institution.

20. APPENDIX A — INSTALLATION & EXECUTION

Prerequisites: Python 3.10 or newer and an internet connection for the first dependency installation.

1. Extract the project ZIP file.
2. Open the expense_tracker folder in Visual Studio Code.
3. Open the integrated terminal.
4. Create a virtual environment: python -m venv .venv
5. Activate it on Windows: .venv\Scripts\activate
6. Install packages: pip install -r requirements.txt
7. Start the app: streamlit run app.py
8. Open the local address printed in the terminal, normally http://localhost:8501.

On first launch, the application creates expenses.db automatically. Use Add Expense to enter test records. Open Manage Budget to set limits. Return to Dashboard to view charts and warnings. Use Transactions to search, download or delete entries.

If the command “streamlit” is not recognised, confirm that the virtual environment is activated and rerun the installation command. If the port is already in use, Streamlit will offer another local port. Do not upload the local database if it contains personal information.

21. APPENDIX B — PUBLISHING TO GITHUB

1. Sign in to GitHub and create a new repository named pennywise-expense-tracker.
2. Choose Public if the assignment requires a public submission. Do not add a licence or README during creation if these files already exist locally.
3. Open the project folder in the terminal.
4. Run: git init
5. Run: git add .
6. Run: git commit -m "Initial PennyWise expense tracker project"
7. Run: git branch -M main
8. Add the remote: git remote add origin https://github.com/YOUR-USERNAME/pennywise-expense-tracker.git
9. Push: git push -u origin main
10. Refresh the repository page and verify app.py, requirements.txt, README.md and docs/project-report.pdf are visible.

The .gitignore file excludes the virtual environment, Python cache files, secrets and the local SQLite database. Never commit passwords, API keys, private customer data or database files containing personal information. Include screenshots in a separate screenshots folder if your faculty asks for proof of execution.

22. APPENDIX C — VIVA QUESTIONS

1. What is the purpose of an expense tracker? — To record and analyse expenditure.
2. Why is Python used? — It is readable and has libraries for interfaces, data and charts.
3. What is SQLite? — A lightweight file-based relational database.
4. What is Streamlit? — A Python framework for creating interactive data applications.

5. What is a DataFrame? — A two-dimensional labelled data structure in Pandas.
6. Why use categories? — They help identify where money is spent.
7. What is a primary key? — A column that uniquely identifies a table row.
8. What is a parameterised query? — SQL that passes values separately from the query text.
9. How is monthly spending calculated? — Filter records by month and sum the amount column.
10. How is budget utilisation calculated? — Actual category spending divided by its budget limit.
11. Why cap the progress bar at 100%? — The visual bar has a maximum while the text reports overspending.
12. What is CSV? — A plain-text tabular data format supported by spreadsheet applications.
13. What are the limitations? — Local, single-user, manual entry and no bank integration.
14. What would you improve? — Authentication, cloud backup, recurring expenses and automated tests.
15. How do you run the project? — Install requirements and execute streamlit run app.py.

23. DETAILED USE-CASE WALKTHROUGH

Scenario: A student receives a monthly allowance and wants to control discretionary spending. The student opens PennyWise and records each purchase on the day it occurs. The following sample transactions illustrate how the application stores and analyses information.

Date	Description	Category	Amount (■)	Payment
01-Nov-2026	College lunch	Food	120	UPI
02-Nov-2026	Bus pass	Transport	450	UPI
03-Nov-2026	Notebook	Education	180	Cash
04-Nov-2026	Movie ticket	Entertainment	250	Card
05-Nov-2026	Snacks	Food	80	Cash
06-Nov-2026	Mobile recharge	Bills	299	UPI

Example calculations: Food spending is ■120 + ■80 = ■200. Total spending in the sample is ■1,379. If the Food budget is ■250, utilisation is ■200 ÷ ■250 × 100 = 80%, so PennyWise displays a warning. If Transport has a budget of ■400, spending of ■450 exceeds the limit by ■50.

Use-case sequence: (1) The user opens Add Expense. (2) The user enters the description, amount, category, date and payment method. (3) The form validates the required description and positive amount. (4) A parameterised SQL INSERT stores the record. (5) The dashboard reloads the stored records and recalculates totals. (6) The user sets category budgets. (7) The dashboard compares actual spending with each limit. (8) The user searches or exports transactions when needed.

Expected benefit: The student can see which categories consume the allowance and take action before the month ends. The system supports awareness and planning; it does not automatically prevent a purchase or access a bank account.

24. EVALUATION, MAINTENANCE & ACCEPTANCE CRITERIA

The project should be evaluated against observable acceptance criteria rather than appearance alone. A successful demonstration should show that data is saved persistently, calculations are correct and invalid input is handled without crashing the application.

Area	Acceptance criterion	Evidence
Expense entry	A valid expense is saved and appears in history.	Form and transaction table
Validation	Blank title and non-positive amount are rejected.	Validation messages
Persistence	Saved records remain after app restart.	Restart demonstration
Analytics	Category and daily totals match sample records.	Dashboard charts
Budget logic	80% warning and over-limit message work.	Budget status panel
Search/export	Search filters rows and CSV downloads.	Filtered table and CSV
Deletion	Selected record is removed.	Updated transaction table

Maintenance recommendations: Keep a backup copy of the SQLite database before making schema changes. Update dependencies deliberately and test the application after each update. Keep source code, documentation and requirements in version control, but exclude the local database, virtual environment and secrets. When a defect is found, reproduce it with a small test dataset, correct the relevant function and repeat the affected test cases.

Project acceptance checklist: The application launches from the documented command; a user can add, view, search, delete and export expenses; budgets can be created and updated; monthly totals and charts are consistent with stored data; empty states are readable; and the GitHub repository contains setup instructions without private data.

Academic demonstration tip: Prepare a small set of fictional transactions and budgets before the viva. Explain one example from input to database row to dashboard total. Be ready to describe the purpose of SQLite, Pandas, Streamlit, Plotly, primary keys, parameterised SQL and the budget-utilisation formula.